

Example 1: Delivery Slot Conflict Detector

(Difficulty 1/5 • Patterns: Intervals, Sorting, Linear Scan)

- **Warm-up (5–7 mins):**

“A delivery company receives reserved loading-dock slots as **[start_time, end_time)**. Write a function that returns **True** if any two reservations conflict and **False** otherwise. Assume a reservation ending exactly when another begins does **not** overlap.”

- **Expected Solution:** Sort reservations by **start_time**. Scan once and compare each start with the previous interval's end. A conflict exists when **current.start < previous.end**.

- **Core Problem (10–15 mins):**

“Return the **first conflicting pair of reservation IDs**, not just a boolean. Preserve the original IDs even though you sort the intervals.”

- **Optimal Approach:**

- Store each interval as **(start, end, id)** before sorting.
- After sorting by **start**, the first conflict can only be with the immediately previous interval if you track the interval with the farthest end seen so far.
- Return the two original IDs; otherwise return “no conflict.”
- **Complexity:** **$O(N \log N)$** time, **$O(N)$** auxiliary space if you copy/sort records.

- **Follow-up / Deep Dive:**

“How would your answer change if intervals were closed **[start, end]** instead of half-open **[start, end)**? What edge cases would you test?”

- **Signal:** Tests whether you define endpoint semantics before coding and recognize the standard interval-overlap invariant.

Example 2: Server Maintenance Window Merger

(Difficulty 1/5 • Patterns: Intervals, Sorting, Merge)

- **Warm-up (5–7 mins):**

“You are several time ranges (windows) for when a server is being maintained. Merge all overlapping or touching windows and return the smallest set of non-overlapping windows.”

- **Expected Solution:** Sort by start time, then build an output list. If the next window begins before or at the current merged end, extend the end; otherwise start a new merged window.

- **Core Problem (10–15 mins):**

*“Also return the **total amount of downtime** after merging, so overlapping maintenance does not get double-counted.”*

- **Optimal Approach:**

- Merge intervals in one scan after sorting.
- Whenever a merged interval closes, add **end - start** to the downtime total.
- Be explicit about whether touching intervals such as **[1,3)** and **[3,5)** should merge.
- **Complexity:** **$O(N \log N)$** time and **$O(N)$** output space.

- **Follow-up / Deep Dive:**

“What if the input is already sorted by start time? What if the windows arrive one by one and you need to maintain the merged result online?”

- **Signal:** Builds comfort with interval normalization, edge conditions, and turning a boolean interval problem into aggregation.

Example 3: Minimum Checkout Kiosks

(Difficulty 1/5 • Patterns: Min-Heap, Resource Allocation, Intervals)

- **Warm-up (5–7 mins):**

*“A store has checkout reservations represented as **[start_time, end_time)**. Return the minimum number of checkout kiosks required so every reservation can run without waiting.”*

- **Expected Solution:** Sort by start time and keep a min-heap of active end times. Reuse a kiosk whenever the smallest end time is $\leq$ **current.start**; otherwise allocate another.

- **Core Problem (15–20 mins):**

*“Return both the minimum kiosk count and the **peak time range** during which that many kiosks are simultaneously busy.”*

- **Optimal Approach:**

- Use the heap solution for the minimum resource count.
- To identify the peak range precisely, a sweep-line representation of **(+1 at start, -1 at end)** is often simpler.
- Process end events before start events at the same timestamp for half-open intervals.
- **Complexity:** $O(N \log N)$ time, $O(N)$ space.

- **Follow-up / Deep Dive:**

“Can you solve only the count using two sorted arrays of starts and ends instead of a heap? Compare the two approaches.”

- **Signal:** Reinforces the classic meeting-rooms pattern and introduces sweep-line reasoning.

Example 4: Assign Actual Resource IDs

(Difficulty 1/5 • Patterns: Two Heaps, Greedy Allocation)

- **Warm-up (5–10 mins):**

“Given reservations [start, end) for identical test machines, assign a machine ID to each reservation while using the minimum possible number of machines.”

- **Expected Solution:** Sort reservations by start. Track active machines in a min-heap ordered by (end_time, machine_id). When machines become free, move their IDs into a second min-heap of reusable IDs.

- **Core Problem (15–20 mins):**

*“Use the **smallest available machine ID** whenever multiple machines are free, and return assignments in the original input order.”*

- **Optimal Approach:**

- Attach each reservation's original index before sorting.
- Release every active machine with **end <= current.start**, not just one.
- Choose the smallest reusable ID; if none exists, create a new ID.
- Write the chosen ID into an answer array at the reservation's original index.
- **Complexity:** $O(N \log N)$ time, $O(N)$ space.

- **Follow-up / Deep Dive:**

“What changes if machines have different capabilities and each reservation requires a minimum capability level?”

- **Signal:** Tests careful state management across multiple heaps and preservation of original order.

Example 5: Recent Request Counter (Did this and prevs on Sep 4- 5)

(Difficulty 1/5 • Patterns: Queue, Sliding Window)

- Warm-up (5–7 mins):

*“Implement **record(timestamp)** for a service that receives timestamps in nondecreasing order. Return how many requests occurred in the inclusive window **[timestamp - 59, timestamp]**.”*

- **Expected Solution:** Keep a deque of timestamps. Push the new timestamp and pop from the front while a timestamp is older than **timestamp - 59**.

- Core Problem (10–15 mins):

*“Change the API to **allow(timestamp, limit) -> bool**, where a rejected request should **not** be recorded.”*

- Optimal Approach:

- First evict expired timestamps.
- If the deque size is already **limit**, reject without pushing.
- Otherwise push the timestamp and accept.
- Every timestamp enters and leaves the deque once, so operations are amortized constant time.
- **Complexity:** amortized **O(1)** per call and **O(limit)** space if rejected events are not stored.

- Follow-up / Deep Dive:

“What assumption makes the deque solution work? What breaks if timestamps can arrive out of order?”

- **Signal:** Teaches the queue-based sliding-window template and the importance of monotonic input assumptions.

Example 6: Burst-Compressed Rate Limiter

(Difficulty 1/5 • Patterns: Sliding Window, Run-Length Compression, Circular Buckets)

- **Warm-up (5–10 mins):**

“A rate limiter may receive millions of events with the same second-level timestamp. Modify the rolling 60-second limiter so it does not store one integer per accepted event.”

- **Expected Solution:** Store (timestamp, count) pairs in a deque and maintain a running total, or use 60 circular buckets indexed by $\text{timestamp} \% 60$.

- **Core Problem (15–20 mins):**

“Implement the fixed-size circular-bucket version. Each bucket must be safely reused after 60 seconds without accidentally counting stale data.”

- **Optimal Approach:**

- Keep arrays `bucket_time[60]` and `bucket_count[60]`.
- For timestamp `t`, index $i = t \% 60$. If `bucket_time[i] != t`, reset that bucket before use.
- To answer the current window count, sum only buckets whose recorded timestamp is within the last 60 seconds, or maintain a carefully updated running total.
- **Complexity:** $O(1)$ space relative to event volume; $O(60)$ query time or $O(1)$ with extra bookkeeping.

- **Follow-up / Deep Dive:**

“How would you make this thread-safe without placing one global lock around every request?”

- **Signal:** Moves from a simple queue to bounded-memory production-style rate limiting.

Example 7: Filesystem Subtree Size with Bad References

(6-7 finished on Sep 6)

(Difficulty 2/5 • Patterns: DFS/BFS, Hash Map, Defensive Traversal)

- **Warm-up (5–10 mins):**

*“Files and directories are stored as nodes with **id**, **size**, and **child_ids**. Given **root_id**, return the total file size reachable beneath that root.”*

- **Expected Solution:** Build or use an ID-to-node lookup map, then perform DFS or BFS and sum each visited node's file size.

- **Core Problem (15–20 mins):**

“The dataset may contain a missing child ID or the same node may be referenced twice. Return an error for missing references and ensure a node's size is counted at most once.”

- **Optimal Approach:**

- Use an explicit **visited** set keyed by node ID.
- Before following a child ID, verify it exists in the lookup map.
- If already visited, skip it for size accounting; optionally report the duplicate reference separately depending on the contract.
- Prefer iterative traversal if depth is unbounded.
- **Complexity:** $O(V + E)$ time and $O(V)$ space.

- **Follow-up / Deep Dive:**

“If duplicate references represent hard links and should count toward physical storage only once but logical directory size multiple times, how would the definition change?”

- **Signal:** Tests graph-vs-tree awareness, defensive lookup behavior, and visited-set semantics.

Example 8: Validate an Organization Hierarchy(hardest problem so far)

(Difficulty 2/5 • Patterns: Graph Validation, In-Degree, DFS/BFS)

- **Warm-up (5–10 mins):**

“Employees are nodes, and each manager lists direct-report IDs. Determine whether the data forms one valid rooted hierarchy.”

- **Expected Solution:** A valid tree has exactly one in-degree-0 root, every other node has in-degree 1, all references exist, no cycle exists, and all nodes are reachable from the root.

- **Core Problem (15–25 mins):**

*“Return a structured reason for failure: **MISSING_NODE**, **MULTIPLE_PARENTS**, **MULTIPLE_ROOTS**, **CYCLE**, or **DISCONNECTED**.”*

- **Optimal Approach:**

- Build the lookup map and in-degree counts while validating child references.
- Reject any node whose in-degree exceeds 1.
- Find the unique root.
- Traverse from the root and detect revisits/cycles; finally verify the visited count equals the number of nodes.
- Order your validation checks deliberately so the returned error is deterministic.
- **Complexity:** $O(V + E)$ time and $O(V)$ space.

- **Follow-up / Deep Dive:**

“What happens with a hierarchy 500,000 levels deep? Why is recursive DFS risky?”

- **Signal:** Strengthens tree-validation reasoning and forces explicit thinking about invalid graph states.

Example 9: Shortest Service Hop Path

(Difficulty 2/5 • Patterns: BFS, Parent Reconstruction) (done on sep 7)

- **Warm-up (5–10 mins):**

*“Services form an unweighted directed graph. Given service **A** and service **B**, return whether **B** is reachable from **A**.”*

- **Expected Solution:** Use BFS or DFS with a visited set. BFS is preferable if you will later need the fewest-hop path.

- **Core Problem (15–20 mins):**

*“Return the **fewest-hop call chain** from **A** to **B**, including both endpoints.”*

- **Optimal Approach:**

- Run BFS from **A**.
- When first discovering a node, record **parent[child] = current**.
- Stop when **B** is discovered or when the queue empties.
- Reconstruct the path by walking parents backward from **B**, then reverse it.
- **Complexity:** $O(V + E)$ time and $O(V)$ space.

- **Follow-up / Deep Dive:**

“If the graph is enormous and you know both endpoints, when might bidirectional BFS help?”

- **Signal:** Practices the most reusable BFS template: queue, visited set, distance/parent tracking.

Example 10: LRU Metadata Cache

(Difficulty 2/5 • Patterns: Hash Map, Doubly Linked List, $O(1)$ Design)

- **Warm-up (5–10 mins):**

*“Design a cache with fixed capacity supporting **get(key)** and **put(key, value)**. When full, evict the least recently used key.”*

- **Expected Solution:** Use a hash map from key to list node plus a doubly linked list ordered from most-recently-used to least-recently-used.

- **Core Problem (20–25 mins):**

“Implement updates correctly: reading a key makes it most recent; overwriting an existing key changes its value and recency without increasing size.”

- **Optimal Approach:**

- Use sentinel head/tail nodes to simplify insert/remove edge cases.
- **get:** lookup node, detach it, move it to the MRU end.
- **put:** update-and-move if present; otherwise insert a new node and evict from the LRU end if over capacity.
- Keep map and list mutations consistent so no stale node remains reachable.
- **Complexity:** $O(1)$ average time per operation and $O(\text{capacity})$ space.

- **Follow-up / Deep Dive:**

“How would you add a byte-size capacity where entries have different sizes instead of a simple item count?”

- **Signal:** Classic interview exercise for combining two data structures to meet an explicit complexity target.

Example 11: TTL Cache with Lazy Expiration

(Difficulty 2/5 • Patterns: Hash Map, Min-Heap, Lazy Deletion)

- **Warm-up (5–10 mins):**

*“Extend a key-value cache so each **put(key, value, expires_at)** has an expiration time. **get(key, now)** must never return expired data.”*

- **Expected Solution:** A hash map alone is enough for correct **get**, but efficient cleanup of expired keys can use a min-heap ordered by expiration time.

- **Core Problem (20–25 mins):**

“Support overwriting the same key many times without needing to delete arbitrary old heap entries.”

- **Optimal Approach:**

- Store the current cache record as **(value, expires_at, generation)** in the map.
- Push **(expires_at, key, generation)** into the min-heap for every write.
- During cleanup, pop expired heap entries; delete from the map only if the generation still matches the current record.
- Old heap entries become harmless stale records.
- **Complexity:** **$O(\log N)$** writes/cleanup pops and **$O(1)$** average map lookup.

- **Follow-up / Deep Dive:**

“How would you prevent the heap from growing forever if a hot key is overwritten millions of times before old heap records expire?”

- **Signal:** Introduces lazy deletion/generation tokens, a pattern that appears in schedulers, leases, and caches.

Example 12: Per-Tenant Sliding Window Limiter

(Difficulty 2/5 • Patterns: Hash Map of Deques, Garbage Collection)

- **Warm-up (10 mins):**

*“Implement a 60-second request limiter where each **tenant_id** has its own independent limit. Timestamps arrive in nondecreasing order globally.”*

- **Expected Solution:** Use a hash map from tenant ID to a deque of accepted timestamps. Evict expired timestamps from that tenant’s deque on each request.

- **Core Problem (20–25 mins):**

“Millions of tenants may appear once and never return. Prevent the tenant map from retaining empty state forever.”

- **Optimal Approach:**

- After evicting old timestamps, erase a tenant entry if its deque becomes empty and no new event is accepted.
- For more proactive cleanup, track tenant expiry times in a secondary min-heap and use lazy generation checks.
- Be clear about whether rejected events affect future limits.
- **Complexity:** amortized $O(1)$ per event for the basic version; additional $O(\log T)$ if maintaining global expiry cleanup.

- **Follow-up / Deep Dive:**

“How would you shard this limiter across CPU cores to reduce lock contention while keeping each tenant’s requests consistent?”

- **Signal:** Takes a known sliding-window technique and adds lifecycle/memory-management concerns.

Example 13: Duplicate Log Suppression Window

(Difficulty 2/5 • Patterns: Hash Map, Queue, Expiration)

- **Warm-up (10 mins):**

“A logging service should emit a message only if the same exact message has not been emitted in the previous 10 seconds. Timestamps are nondecreasing.”

- **Expected Solution:** Track the last emitted timestamp per message. Emit if absent or sufficiently old.

- **Core Problem (20 mins):**

“The set of possible messages is unbounded. Remove old message entries so memory depends on the active 10-second window, not total history.”

- **Optimal Approach:**

- Keep **last_emitted[message] = timestamp** plus a queue of **(timestamp, message)** records.
- Before processing a new event, pop queue records older than the retention window.
- Delete a map entry only if its stored timestamp equals the popped record; otherwise that queue record is stale because the message was emitted again later.
- **Complexity:** amortized **O(1)** average time and **O(active distinct messages)** space.

- **Follow-up / Deep Dive:**

“How would you support a per-message suppression interval instead of one global 10-second value?”

- **Signal:** Reuses the stale-record technique in a simpler setting and teaches bounded-state stream processing.

Example 14: Top-K Error Codes in a Rolling Window

(Difficulty 2/5 • Patterns: Sliding Window, Frequency Map, Heap)

- **Warm-up (10 mins):**

*“Given a finite list of error events (**timestamp**, **error_code**) sorted by timestamp, return the **K** most frequent error codes in the final 5-minute window.”*

- **Expected Solution:** Use two pointers or a deque to identify the active window, maintain a frequency map, then select top K with a heap.

- **Core Problem (20–30 mins):**

*“Events arrive continuously. Support **record(event)** and **top_k(now, K)** while expiring old events exactly.”*

- **Optimal Approach:**

- Maintain a deque of active events and a frequency map.
- On each record/query, evict events older than the window and decrement counts.
- For a straightforward interview solution, build a size-K min-heap from the current frequency map when **top_k** is called.
- Discuss the tradeoff: fast writes + more expensive queries versus maintaining a more complex ordered frequency structure.
- **Complexity:** updates amortized $O(1)$; query $O(D \log K)$ for **D** active distinct codes.

- **Follow-up / Deep Dive:**

*“What would you do if **top_k** is called thousands of times per second but writes are relatively rare?”*

- **Signal:** Forces a workload-based data-structure tradeoff instead of blindly optimizing one operation.

Example 15: Build Dependency Cycle Detector

(Difficulty 3/5 • Patterns: Directed Graph, DFS Coloring, Kahn's Algorithm)

- **Warm-up (10 mins):**

*"Build targets depend on other targets. Given directed edges **A -> B** meaning "A depends on B," determine whether the dependency graph contains a cycle."*

- **Expected Solution:** Use DFS with three colors/states or Kahn's topological-sort algorithm. A back-edge to a currently visiting node indicates a cycle.

- **Core Problem (20–25 mins):**

*"If a cycle exists, return **one concrete cycle path** such as **A -> B -> C -> A**."*

- **Optimal Approach:**

- DFS is convenient for cycle reconstruction.
- Track each node as unvisited, visiting, or done, and keep a parent pointer.
- When an edge reaches a visiting node, walk parent pointers to reconstruct the cycle.
- Handle disconnected components by starting DFS from every unvisited node.
- **Complexity:** $O(V + E)$ time and $O(V)$ space.

- **Follow-up / Deep Dive:**

"How would an iterative implementation avoid recursion depth problems on a graph with millions of nodes?"

- **Signal:** Moves beyond detecting a bad condition to producing a useful diagnostic witness.

Example 16: Deterministic Deployment Order

(Difficulty 3/5 • Patterns: Topological Sort, Min-Heap)

- **Warm-up (10 mins):**

“Services must be deployed after all of their dependencies. Return any valid deployment order, or report that no valid order exists.”

- **Expected Solution:** Use Kahn’s algorithm: compute in-degrees, enqueue zero-in-degree nodes, remove them, and decrement dependents.

- **Core Problem (20–25 mins):**

“When several services are simultaneously deployable, always choose the lexicographically smallest service name so the output is deterministic.”

- **Optimal Approach:**

- Use a min-heap instead of a FIFO queue for zero-in-degree nodes.
- Pop the smallest available service, append it to the output, and release its dependents.
- If fewer than **V** nodes are output, a cycle exists.
- **Complexity:** $O((V + E) \log V)$ with the heap.

- **Follow-up / Deep Dive:**

*“How would you group services into deployment **waves**, where every service in a wave can deploy in parallel?”*

- **Signal:** Builds a strong topological-sort template and shows how ordering requirements change the supporting data structure.

Example 17: Parallel Build Scheduler with K Workers

(Difficulty 3/5 • Patterns: Topological Sort, Event Simulation, Heaps)

- **Warm-up (10 mins):**

“Each build target has a duration and dependencies. With unlimited workers, compute the earliest finish time of every target.”

- **Expected Solution:** Topologically process the DAG and use dynamic programming: $\text{earliest_start}[v] = \max(\text{earliest_finish}[\text{parent}])$.

- **Core Problem (25–35 mins):**

“Now there are only K identical workers. Simulate execution and return the total build completion time. When multiple ready tasks exist, choose the smallest task ID.”

- **Optimal Approach:**

- Maintain in-degrees and a min-heap of ready task IDs.
- Maintain another min-heap of running tasks ordered by $(\text{finish_time}, \text{task_id})$.
- At each moment, start ready tasks while workers are free; then advance time to the next completion and release newly ready dependents.
- Process all tasks finishing at the same timestamp before starting the next batch.
- **Complexity:** $O((V + E) \log V)$ for the simulation.

- **Follow-up / Deep Dive:**

“Why does a simple greedy tie-break not necessarily minimize total makespan for arbitrary DAGs and durations?”

- **Signal:** Combines topological ordering with discrete-event simulation and tests precision around simultaneous events.

Example 18: Lowest-Latency Service Route

(Difficulty 3/5 • Patterns: Dijkstra, Priority Queue, Path Reconstruction)

- **Warm-up (10 mins):**

*“Services form a directed graph where each edge has nonnegative network latency. Return the minimum latency from source service **S** to target **T**.”*

- **Expected Solution:** Use Dijkstra’s algorithm with a min-priority queue and a distance map/array.

- **Core Problem (20–30 mins):**

“Return both the minimum total latency and the actual service path. The graph may contain multiple stale priority-queue entries for the same node.”

- **Optimal Approach:**

- Initialize **dist[S] = 0** and push **(0, S)**.
- When popping, skip the entry if its distance no longer equals the best known **dist[node]**.
- Relax outgoing edges and store **parent[next] = node** on improvement.
- Stop when **T** is popped with its current best distance.
- **Complexity:** $O((V + E) \log V)$ with adjacency lists and a binary heap.

- **Follow-up / Deep Dive:**

“Why does Dijkstra fail with negative edge weights? What algorithm would you consider instead?”

- **Signal:** Practices the essential “stale heap entry” pattern and shortest-path reconstruction.

Example 19: Dynamic Network Connectivity

(Difficulty 3/5 • Patterns: Union-Find / DSU)

- **Warm-up (10 mins):**

*“Servers begin disconnected. You receive **connect(a, b)** operations and must answer **same_cluster(a, b)** queries efficiently.”*

- **Expected Solution:** Use disjoint-set union with path compression and union by rank/size.

- **Core Problem (20–25 mins):**

“After each connection, also return the size of the connected cluster containing a given server and the total number of clusters.”

- **Optimal Approach:**

- Initialize each server as its own parent with size 1.
- **find** compresses paths; **union** attaches the smaller tree under the larger root.
- When two different roots merge, add their sizes and decrement the component count.
- **Complexity:** nearly **$O(1)$** amortized per operation, formally **$O(\alpha(N))$** .

- **Follow-up / Deep Dive:**

“What if links can also be deleted? Why is ordinary DSU no longer enough for fully dynamic connectivity?”

- **Signal:** Introduces a high-value data structure and clarifies exactly which dynamic operations it supports well.

Example 20: Nearest Healthy Replica

(Difficulty 3/5 • Patterns: Multi-Source BFS, Graph Distances)

- **Warm-up (10 mins):**

"In an unweighted network, some nodes host healthy replicas. For one client node, return the distance to the nearest healthy replica."

- **Expected Solution:** A BFS from the client works for one query.

- **Core Problem (20–30 mins):**

*"There may be millions of client queries but the healthy replica set changes rarely. Precompute the nearest-replica distance for **every** node."*

- **Optimal Approach:**

- Run one multi-source BFS initialized with all healthy replica nodes at distance 0.
- Expand outward once; the first time a node is visited is its distance to the closest replica.
- Optionally store which replica "owns" each node by propagating the source ID.
- **Complexity:** $O(V + E)$ preprocessing and $O(1)$ distance queries.

- **Follow-up / Deep Dive:**

"How would the solution change for weighted network links? What if healthy replicas are added frequently?"

- **Signal:** Teaches the powerful reversal from many single-source queries to one multi-source traversal.

Example 21: Versioned Configuration Store

(Difficulty 3/5 • Patterns: Hash Map, Sorted Vectors, Binary Search)

- **Warm-up (10 mins):**

*“Implement **set(key, value, timestamp)** and **get(key, timestamp)**, which returns the value written at the greatest timestamp $\leq$ the requested timestamp. Writes for each key arrive in increasing timestamp order.”*

- **Expected Solution:** Map each key to a vector of **(timestamp, value)** records and binary-search the vector on **get**.

- **Core Problem (20–25 mins):**

*“Also support **get_range(key, start, end)** returning all versions whose timestamps lie in the closed interval **[start, end]**.”*

- **Optimal Approach:**

- Use **lower_bound** to locate the first timestamp $\geq$ **start**.
- Use **upper_bound** to locate the first timestamp $>$ **end**.
- The slice between those positions is exactly the requested range.
- For point lookup, use **upper_bound(timestamp)** and step back one position if possible.
- **Complexity:** **$O(1)$** amortized append, **$O(\log M)$** point lookup, and **$O(\log M + R)$** range query.

- **Follow-up / Deep Dive:**

“What breaks if writes for a key can arrive out of timestamp order? Compare sorted insertion versus buffering and periodic compaction.”

- **Signal:** Builds binary-search fluency in a realistic versioned-data setting.

Example 22: Capacity-Limited Reservation Calendar

(Difficulty 4/5 • Patterns: Sweep Line, Ordered Map, Prefix Sum)

- **Warm-up (10 mins):**

*“A lab has capacity C . Given a **batch** of reservations $[start, end)$, determine whether the total number of simultaneous reservations ever exceeds C .”*

- **Expected Solution:** Create delta events $+1$ at starts and -1 at ends, sort by time, and scan a running prefix sum.

- **Core Problem (25–35 mins):**

*“Implement **book**(start, end) online. Accept a booking only if adding it would never make concurrent usage exceed C ; rejected bookings must leave the data structure unchanged.”*

- **Optimal Approach:**

- Maintain an ordered map of time deltas. Tentatively add $+1$ at start and -1 at end.
- Scan the ordered deltas and track the running count; reject if it exceeds C .
- If rejected, roll back both delta changes and erase zero-valued map entries.
- This straightforward version is interview-friendly but can be $O(T)$ per booking for T distinct timestamps.
- **Complexity:** $O(T)$ validation per booking with an ordered map; discuss stronger structures later.

- **Follow-up / Deep Dive:**

“How would you improve repeated booking checks to roughly $O(\log T)$ using an augmented balanced tree or segment tree?”

- **Signal:** Transitions from static sweep-line logic to dynamic state updates and rollback.

Example 23: Peak Concurrent Connections

(Difficulty 4/5 • Patterns: Sweep Line, Event Ordering, Aggregation)

- **Warm-up (10 mins):**

“Connection sessions are [connect_time, disconnect_time). Return the maximum number of simultaneous connections.”

- **Expected Solution:** Use start/end deltas and a sweep line, or sort start and end arrays separately.

- **Core Problem (20–30 mins):**

“Return all maximal time ranges during which the system is at the peak concurrency, merging adjacent peak ranges where appropriate.”

- **Optimal Approach:**

- Aggregate all deltas at the same timestamp before moving to the next distinct time.
- For half-open intervals, the active count after applying the timestamp's combined delta is the count on [time, next_time).
- Track each segment whose active count equals the global maximum.
- A clean approach is two passes: first find the peak value, second emit peak segments; or store candidate segments in one pass and update carefully.
- **Complexity:** $O(N \log N)$ time and $O(N)$ event space.

- **Follow-up / Deep Dive:**

“How does event ordering change if sessions use closed intervals? Why can processing individual same-time events in the wrong order produce an off-by-one error?”

- **Signal:** Sharpens endpoint semantics and sweep-line implementation details that often cause interview bugs.

Example 24: Out-of-Order Event Window Aggregator

(Difficulty 4/5 • Patterns: Min-Heap, Watermarks, Streaming)

- **Warm-up (10 mins):**

*“Events have an **event timestamp** but may arrive slightly out of order. Assume an event can arrive at most 5 seconds late. Compute counts for completed 60-second windows.”*

- **Expected Solution:** Track the maximum event timestamp seen. A safe watermark is **max_seen - 5**; windows ending at or before that watermark can be finalized.

- **Core Problem (25–35 mins):**

“Implement an aggregator that accepts events, buffers only what is necessary, and emits finalized per-second counts in timestamp order.”

- **Optimal Approach:**

- Keep **max_seen_timestamp**.
- Buffer not-yet-safe events in a min-heap ordered by event timestamp, or place them directly into bounded timestamp buckets.
- Advance the watermark as newer timestamps arrive. Pop/process events whose timestamps are now safe.
- Finalize windows/buckets only when no future legal event can still belong to them.
- **Complexity:** depends on lateness bound; memory is bounded by events/buckets within the lateness horizon rather than total history.

- **Follow-up / Deep Dive:**

*“What policy would you use for events that arrive **later** than the promised 5-second bound: drop, correct previous output, or send a compensating update?”*

- **Signal:** Introduces event-time versus processing-time reasoning, a core streaming-systems concept.

Example 25: Fixed-Memory Metrics Aggregator

(Difficulty 4/5 • Patterns: Circular Buffer, Time Buckets, Rolling Aggregates)

- **Warm-up (10 mins):**

“Track the number of requests and sum of latencies for the last 10 minutes using one-second granularity. Event timestamps are nondecreasing.”

- **Expected Solution:** Use a 600-slot circular array. Each slot stores its exact second, count, and latency sum so stale slots can be detected when reused.

- **Core Problem (25–30 mins):**

*“Support **record(timestamp, latency)** and **query(now)** returning request count, average latency, and requests per second over the last 10 minutes.”*

- **Optimal Approach:**

- Index a second with **timestamp % 600**. Reset the slot if its stored timestamp differs.
- Maintain running totals to make queries constant time, but subtract a slot’s old contribution whenever that slot is reset.
- If time jumps forward by more than the whole window, clear/reset all aggregate state safely.
- Guard division by zero for average latency.
- **Complexity:** **O(1)** record and query time, **O(600)** fixed space.

- **Follow-up / Deep Dive:**

“How would you add p95 latency? Why are exact percentiles harder to maintain in fixed memory than counts and sums?”

- **Signal:** Tests circular-buffer correctness, stale bucket handling, and awareness of mergeable versus non-mergeable metrics.

Example 26: Consistent-Hash Cache Router

(Difficulty 4/5 • Patterns: Ordered Map, Hashing, Virtual Nodes)

- **Warm-up (10 mins):**

“Given a sorted ring of server hash positions, route a key to the first server position clockwise from the key’s hash, wrapping around at the end.”

- **Expected Solution:** Use **lower_bound** on the sorted ring. If it returns end, wrap to the first server.

- **Core Problem (25–35 mins):**

“Support adding and removing servers using multiple virtual nodes per server, then route keys in $O(\log V)$ where V is the number of virtual nodes.”

- **Optimal Approach:**

- Hash each **(server_id, vnode_index)** to a ring position and store position -> server in an ordered structure.
- Lookup uses **lower_bound(hash(key))** with wraparound.
- On add/remove, insert or erase only that server’s virtual-node positions.
- Handle hash collisions explicitly, e.g. map a position to a small ordered list or use a compound key **(hash, vnode_id)**.
- **Complexity:** routing $O(\log V)$; add/remove $O(R \log V)$ for R virtual nodes per server.

- **Follow-up / Deep Dive:**

“Why do virtual nodes improve balance? How would you measure how many keys move when one server is removed?”

- **Signal:** Bridges interview data structures with a genuine distributed-systems partitioning primitive.

Example 27: Renewable Lease Manager

(Difficulty 4/5 • Patterns: Min-Heap, Hash Map, Generation Tokens)

- **Warm-up (10 mins):**

*“Implement leases for resources. **acquire(resource, owner, expires_at)** succeeds only if the resource is currently unleased or expired. **release** frees it early.”*

- **Expected Solution:** Keep current lease state in a hash map and expiration candidates in a min-heap.

- **Core Problem (25–35 mins):**

*“Add **renew(resource, owner, new_expires_at)**. Old expiration heap entries must never expire a newer renewed lease.”*

- **Optimal Approach:**

- Each current lease gets a monotonically increasing **generation** or unique lease token.
- Heap entries are (**expires_at, resource, generation**).
- When popping an expired entry, remove the lease only if the resource still exists and the generation matches.
- **renew** writes a new expiration/generation and pushes a fresh heap record.
- **Complexity:** $O(\log N)$ acquire/renew plus $O(1)$ average current-state lookup.

- **Follow-up / Deep Dive:**

“In a distributed system, why is an owner name alone not enough to prevent an old client from acting after its lease has expired and been reacquired?”

- **Signal:** Reinforces stale-entry handling and introduces fencing-token thinking without requiring a full distributed-systems design.

Example 28: Dependency-Aware Retry Scheduler

(Difficulty 4/5 • Patterns: DAG, Priority Queue, Event Simulation, Backoff)

- **Warm-up (10 mins):**

“Jobs have dependencies. A job becomes runnable only after all dependencies succeed. Given a DAG, return the initial set of runnable jobs.”

- **Expected Solution:** Compute in-degrees; jobs with zero unfinished dependencies are initially runnable.

- **Core Problem (30–40 mins):**

“Simulate a scheduler where failed jobs retry after exponential backoff, successful jobs unlock dependents, and the next runnable action should be chosen by earliest eligible time then smallest job ID.”

- **Optimal Approach:**

- Track each job’s remaining dependency count and terminal status.
- Use a min-heap of runnable attempts ordered by **(eligible_time, job_id, attempt)**.
- On success, decrement each dependent’s remaining count and enqueue it when the count reaches zero.
- On retryable failure, compute the next backoff time and enqueue another attempt.
- Use an attempt/generation field if stale scheduled entries can exist after cancellation or terminal failure.
- **Complexity:** roughly $O((E + A) \log A)$ where **A** is the number of attempts processed.

- **Follow-up / Deep Dive:**

“How would you prevent a dependency failure from causing downstream jobs to wait forever?”

- **Signal:** Combines graph state, scheduling, and lazy invalidation into one realistic systems-coding exercise.

Example 29: Dynamic Range Capacity Reservations

(Difficulty 5/5 • Patterns: Segment Tree, Lazy Propagation, Range Add / Range Max)

- **Warm-up (10–15 mins):**

“Time is discretized into M slots. Each booking consumes one unit of capacity on every slot in $[start, end)$. Given all bookings offline, determine the maximum load on any slot.”

- **Expected Solution:** A difference array plus prefix sum solves the static batch problem in $O(M + N)$.

- **Core Problem (35–45 mins):**

“Now bookings and cancellations arrive online. Support $add(start, end)$, $cancel(start, end)$, and $max_load()$ efficiently for up to hundreds of thousands of operations.”

- **Optimal Approach:**

- Use a segment tree storing the maximum load in each node.
- A booking is a range $+1$; cancellation is a range -1 .
- Lazy propagation lets a full-covered node absorb the delta without descending to every time slot.
- The root's stored maximum answers $max_load()$ immediately.
- Validate cancellations if the API must forbid negative per-slot load; that may require also tracking range minimum.
- **Complexity:** $O(\log M)$ range updates and $O(1)$ global max query after each update.

- **Follow-up / Deep Dive:**

“If time coordinates are huge sparse integers rather than $0..M-1$, how would coordinate compression or a dynamic segment tree help?”

- **Signal:** A serious data-structure step-up: recognizing when sweep-line recomputation is no longer fast enough for dynamic range updates.

Example 30: Mini Distributed Task Scheduler Core

(Difficulty 5/5 • Patterns: DAG, Heaps, Leases, Retries, Idempotency, State Machines)

- **Warm-up (10–15 mins):**

“Jobs form a dependency DAG and each job has a priority. Given all jobs whose dependencies are already satisfied, return the next job to run using highest priority and then smallest job ID as a tie-break.”

- **Expected Solution:** Use a priority queue for ready jobs plus dependency counters maintained from the DAG.

- **Core Problem (40–60 mins):**

*“Design and implement the in-memory core of a scheduler with **submit(job)**, **claim(worker, now)**, **complete(job, lease_token)**, **fail(job, lease_token, now)**, and **heartbeat(job, lease_token, new_expiry)**. Jobs may have dependencies, priority, retries with backoff, and expiring worker leases.”*

- **Optimal Approach:**

- Model each job with an explicit state such as **BLOCKED**, **READY**, **LEASED**, **RETRY_WAIT**, **SUCCEEDED**, **FAILED**.
- Maintain remaining dependency counts and reverse dependency lists so success can unlock children.
- Use a ready priority queue for runnable jobs and a time-ordered heap for retry/lease-expiry events.
- Issue a unique or monotonically increasing **lease token** on every claim; completion/failure/heartbeat must match the current token so stale workers cannot mutate a re-leased job.
- Use generation checks to ignore stale heap events after renewals or state changes.
- Make terminal transitions idempotent: duplicate completion messages should not unlock dependents twice.
- **Complexity:** most queue/state transitions should be **$O(\log N)$** plus **$O(\text{out_degree})$** when a job succeeds.

- **Follow-up / Deep Dive:**

“Walk through these failure cases: worker dies after claim; completion RPC is duplicated; retry timer and late completion race; dependency permanently fails; scheduler process restarts. Which guarantees can an in-memory implementation provide, and what would require durable storage?”

- **Signal:** Capstone problem combining nearly every pattern in the set while testing invariants, failure handling, and clear state-machine design.